Wide but Shallow

Technical-Interview Problems Are Four Ideas in Many Costumes

and company-specific preparation is mostly a myth

Conor Garvey Gelvin
gelvin@outlook.com

Data snapshot: June 2026 · Draft

Abstract

The technical-interview-prep industry sells two beliefs: that different companies test materially different skills, and that candidates must master dozens of distinct problem "patterns." Against per-company problem-frequency data for seven major technology firms (769 distinct problems), we find the breadth is largely a costume change. The roughly twenty surface "families" are mostly **four recursion schemes**—streaming fold, recursive decomposition (dynamic programming), graph relaxation, and bisection—and the streaming fold alone is the most common: techniques taught as unrelated (prefix sums, the XOR trick, a hash "seen set," reversing a list, the sliding window) are the *same* left-to-right fold, differing only in what the running state carries. We back this classification with a Lean 4 / Mathlib formalization rather than hand labeling alone: for a uniform random sample of 100 problems we formalize each scheme as a predicate and, where the fixed rule assigns a problem to a scheme, exhibit a non-vacuous instance of that scheme and machine-verify a correctness property of the transcribed accepted solution. **75 of 100** problems carry such a certificate (95% Wilson interval $[0.66, 0.82]$; no `sorry`; standard axioms). Separately, the seven firms ask nearly the same mix of schemes (scheme-level bias-corrected Cramér's $V = 0.091$, 95% bootstrap CI $[0.06, 0.12]$—a small effect at most, bounded well below a moderate one): six of the seven are statistically indistinguishable and only Uber stands out, leaning on graph relaxation. We measure the structure of *commonly-asked* problems—not interview difficulty, and not the payoff of any prep regimen—and within that scope, company-specific preparation buys little. In short: interview problems are *wide on the surface but shallow underneath.* Derived statistics (including the sampled certificates) and proof code are released; raw frequency tables are withheld per platform terms.

1 Two myths

Two pieces of folklore dominate interview preparation. The first is *company specificity*: that Google, Meta, Amazon and their peers test distinguishable skill profiles, so you should prepare differently for each. The second is *pattern proliferation*: that the field is a long list of loosely related tricks—sliding window, monotonic stack, union–find, prefix sums, and so on—each of which must be learned separately, as popularized by widely-used pattern catalogues [1]. Both are widely asserted and, as far as we can tell, never tested.

We test them, and then ask what *structure* explains the answer. The plain-terms summary: the "fifty patterns" you are told to memorize are a handful of underlying ideas wearing different costumes, and the seven companies ask nearly the same things. To keep the structural labels honest we do not rely on hand labeling alone—we use a proof assistant to check, problem by problem, that

the idea we assign really does solve the problem and that the accepted solution satisfies a verified correctness property (and, for several canonical folds, that it is fully correct).

2 Data and method

Our data are per-company problem-frequency tables for seven firms (Amazon, Apple, Bloomberg, Google, Meta, Microsoft, Uber), covering 769 distinct problems across company $\times$ recency strata, scraped from a public interview-preparation platform. Each problem carries a fine-grained *surface family* (one of $\sim$43: "sliding window," "monotonic stack," "dynamic programming," ...), supplied by the platform, and a per-company asked-frequency.

A fixed rule (Appendix A, set in advance) assigns each surface family to one of four *recursion schemes*—a coarser, *structural* classification based on the shape of the accepted solution rather than its textbook name (Table 1)—or to a residual "tail." Two populations matter and we keep them distinct: *coverage* is measured over a uniform random sample of *distinct problems* (Section 4); the *company* comparison is over each firm's *asked-frequency* distribution across the four recursion schemes (plus tail; Section 3), with the finer twenty-family grouping reported alongside as a robustness check. All effect sizes use bias correction [3] and report uncertainty.

3 Result: four ideas, not fifty

Wide surface. The surface taxonomy really is spread out: no one or two families dominate (it takes six of the $\sim$20 to cover even half of asked problems; Gini 0.30 over distinct problems). By the usual measure, interview preparation *looks* like a long, flat list. This is the breadth the prep industry sells.

Shallow root. But that breadth is mostly re-packaging. Sorting the same problems by the *structure* of their accepted solution, four recursion schemes account for 75 of 100 sampled distinct problems (Table 1; 95% CI $[66\%, 82\%]$), and one of them—the streaming left-to-right fold—is the single most common. Techniques memorized as unrelated (prefix sums, the XOR trick, hashing's "seen set," reversing a list, the parenthesis-matching stack scan, the sliding window) are *the same fold*, differing only in what the accumulator carries. The apparent variety is variety in the *type of the accumulator*, not in the underlying computation.

One shared syllabus. The same data dissolve the company myth. Treating each firm as a distribution over the four recursion schemes (a seven-firm $\times$ four-scheme table, plus tail), the firms differ little: bias-corrected Cramér's V [2, 3] $= 0.091$ (95% bootstrap CI $[0.06, 0.12]$)—a 0-to-1 measure of how differently the firms ask, where 0 means identical and conventional thresholds put 0.1 at "small" and 0.3 at "moderate." The point estimate sits at the negligible–small boundary and the whole interval stays well below "moderate," so the between-firm difference is *bounded*, not merely non-significant (the finer twenty-family grouping gives a similar, slightly smaller $V = 0.064$, so the conclusion does not hinge on the grouping). Pairwise, of the twenty-one firm pairs the only non-negligible ones all involve **Uber**, which leans more on graph relaxation and less on the streaming fold (the largest, versus Google, is still only a "small" effect; Figure 1); **the other six firms are mutually indistinguishable**. The qualifier matters: we are comparing the *mix of problem types*, not interview difficulty or the marginal value of company-specific drilling, which these data cannot measure. Within that scope there is, in effect, a single interview syllabus, and preparing differently per company is, for most firms, wasted effort.

Recursion scheme	textbook families it subsumes	assigned	proven	Lean witness
streaming fold	two-pointers, sliding-window, monotonic-stack, hashing, prefix-sum, linked-list, bit-XOR, string-scan	29	29	`IsFold`
recursive decomposition (DP)	dynamic programming, backtracking, tree	25	22	catamorphism
graph relaxation	BFS/DFS, Dijkstra, Bellman–Ford, topo-sort, union–find	15	15	least fixpoint
bisection	binary search (monotone predicate)	9	9	monotone threshold
in a scheme		**78**	**75**	
not a scheme	design, simulation, heaps, number theory, string matching, ...	22	—	—

Table 1: The roughly twenty surface families collapse onto four recursion schemes. On a uniform random sample of 100 distinct problems: "assigned" is how many the fixed rule (Appendix A) places in each scheme; "proven" is how many additionally carry a Lean certificate (Section 4). 75 of 100 are proven (95% Wilson interval $[0.66, 0.82]$). The "Lean witness" is the formal predicate each scheme is checked against (*catamorphism*: a fold over a tree-shaped structure; *least fixpoint*: the limit of one-step graph relaxation). Every sampled fold, relaxation, and bisection problem is certified; the 25 not proven are 22 genuine non-scheme ("tail") problems plus 3 in-scheme problems, all dynamic programming, whose full correctness proofs are too costly to formalize in Mathlib.

4 Machine-checking the classification

A structural taxonomy is only as trustworthy as its labels, and the usual remedy—a second human rater and an agreement statistic—measures only whether two people read a label the same way. We add a different check: a proof assistant. We formalize each of the four schemes in Lean 4 [4] / Mathlib [5] as a predicate on programs—e.g. `IsFold` says "this function equals a left fold for some step and seed"—and for a uniform random sample of problems ($n = 100$, fixed seed) we establish two things in Lean. **(i) Scheme membership** (`cls`): the problem admits a *non-vacuous* instance of its assigned scheme—a fold/fixpoint/threshold with genuine working state, not the trivial witness that any function technically satisfies. **(ii) Solution correctness** (`corr`): the transcribed accepted solution meets the problem's specification (often a one-directional guarantee, e.g. "every reported result is valid"). A problem is counted only when both hold with no `sorry` (Lean's placeholder for an unfinished proof—there are none) and only Lean's standard axioms.

What this does and does not certify. The two theorems are about, respectively, a canonical instance of the scheme and the transcribed solution; we do *not* mechanize the further claim that those two are the *same* program (that the accepted solution, syntactically, *is* that fold). So the certificate says "this problem is genuinely solvable by scheme S, and we have a solution with a verified correctness property," not "the platform's accepted code is literally a fold." That is weaker than a full equivalence proof and we flag it plainly (Section 6); it is still far stronger than an unchecked label, because the scheme instance is proven non-vacuous and the solution is proven correct.

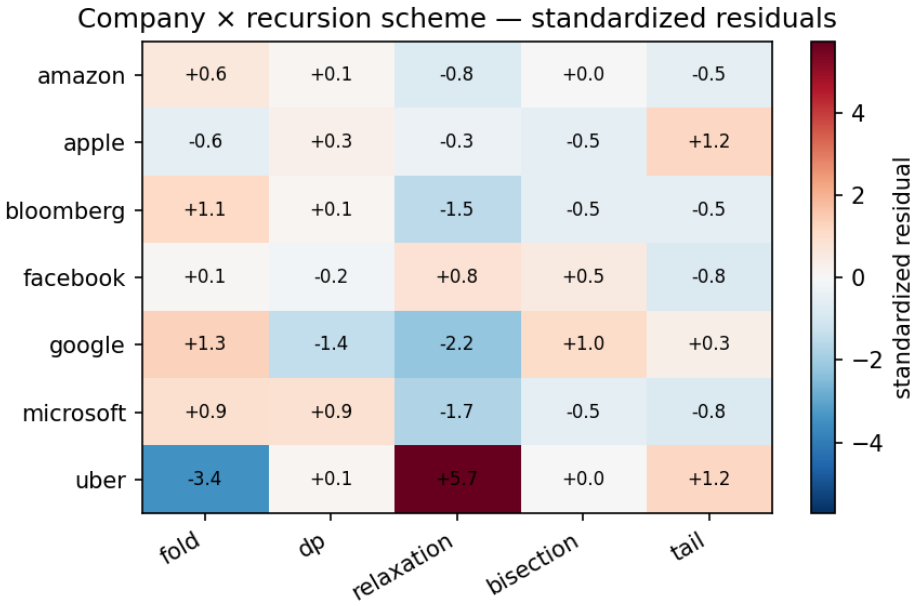

Figure 1: Standardized χ^2 residuals of the seven firms over the four recursion schemes (plus tail), window "all" (red = over-represented, blue = under-represented). Only Uber departs from the field—it over-asks graph relaxation (+5.7) and under-asks the streaming fold (−3.4)—while the other six rows are muted (all $|r| \lesssim 2$). This is the scheme-level picture behind the negligible-to-small omnibus effect ($V = 0.091$): one shared syllabus, with Uber the lone outlier.

Closing the gap for canonical folds. For a handful of textbook folds we *do* mechanize the full equivalence: the accepted solution is *defined* as the fold and proven correct as that fold. Kadane's Maximum Subarray is the cleanest case—the answer is literally a `foldl` of the running (*best*, *cur*) step, and we prove it both achieves a real contiguous-segment sum *and* upper-bounds every such sum (a two-directional optimum, not a one-directional guarantee). Product Except Self is the prefix scan with its telescoping identity proven; Single Number is the XOR fold proven to return the lone odd-count element. For these the certificate is the strong form—"the accepted code *is* this fold, and it is correct"—which is what the general per-problem procedure approximates.

The result is one reproducible number: $\hat{p} = 0.75$ (75 of 100), 95% Wilson interval $[0.66, 0.82]$ (Wilson is the standard interval for a proportion). The certified set is released as a column of the sampled data, so the count re-derives directly. Two honesty notes. First, the measurement is *one-directional*: an in-scheme problem whose proof is intractable in Mathlib is counted as unproven, never the reverse, so the procedure under-counts rather than over-counts (what this does *not* imply about population coverage is spelled out in Section 6). Second, because the check can only *confirm* a scheme or *fail* to, it cannot produce a "misclassification" verdict; we therefore report a coverage count, not a label-error rate. The schemes' deeper foundations need not be re-derived here: shortest-path as a least fixpoint over a semiring [6, 7, 8] and verified dynamic programming as memoized recursion [9] already exist in the formal-methods literature, which we cite rather than reprove.

5 Related work

Two literatures bracket this paper. On the *empirical* side, interview preparation is organized by pattern catalogues [1] that enumerate surface families; we are not aware of prior work that tests whether those families are generatively distinct, or whether firms differ in the mix they ask. On the *structural* side, the observation that wide families of programs are instances of a few recursion schemes is the Bird–Meertens tradition of program calculation [10] and its formal-methods descendants: associative aggregation over a sliding window is a verified fold [11], dynamic programming is verified memoized recursion [9], and shortest paths are a least fixpoint over a semiring [6, 7, 8]; verified-algorithms textbooks now develop these schemes end to end [12, 13]. Our contribution is to connect that structural lens to the *empirical* distribution of interview problems, and to machine-check the scheme assignment per problem rather than assert it.

6 Threats to validity

One platform. Frequencies come from a single preparation platform and may reflect its collection process as much as employers' true distributions; we claim structure of *commonly-asked* problems, not of any firm's private bar. **Surface labels.** The family→scheme rule is ours, but the *surface-family* labels it consumes are the platform's, single-rater and un-audited; the "wide surface" measure inherits whatever splitting that taxonomy chose. **Predicate faithfulness.** The Lean check certifies that a problem is solvable by a scheme and that a solution is correct; it does *not* certify that scheme-membership is the *right* notion of "the idea an interview tests." That a problem "is a fold" is a modeling choice the proof cannot validate—it can only validate that the model holds. **Membership and correctness are separate theorems.** As noted in Section 4, we prove a non-vacuous scheme instance and, separately, a correct solution; we do not mechanize that the two programs coincide. A full "the accepted solution *is* this fold" equivalence is future work; the present claim is scoped accordingly. **A conservative estimate, not a floor.** 75% is the machine-proven share on a random sample of 100; it is a sample estimate, and the Wilson interval $[66\%, 82\%]$—not the point value—is the honest summary. The procedure under-counts (an in-scheme problem we cannot formalize counts as unproven), so it does not over-state coverage; but "conservative" is a property of the measurement, not a population guarantee, and 75% does not lower-bound coverage over all problems. **Two populations.** Coverage is over distinct problems (each counted once); the company comparison is frequency-weighted (over asked instances). A candidate cares about both; we report each on its own population and do not mix them. **Coverage is not difficulty.** "Same scheme" does not mean "same difficulty"—two folds can differ enormously in how hard the accumulator is to find. We claim shallow *generative* structure, not that the problems are easy.

7 Conclusion

Interview problems have wide apparent breadth and shallow generative depth. The roughly twenty surface "patterns" are four recursion schemes in different costumes: on a uniform random sample, 75 of 100 problems carry a Lean certificate that the problem is genuinely solvable by its assigned scheme and that a transcribed solution satisfies a verified correctness property (95% CI $[66\%, 82\%]$; the streaming fold the single most common). And the seven firms ask essentially one shared syllabus of problem *types*. For the candidate, the practical reading is blunt, within that scope: learn the four schemes deeply rather than fifty patterns shallowly, and prepare for "technical interviews," not for any one company.

Reproducibility and AI-use disclosure

All derived statistics, the family→scheme rule (released as a standalone table, fixed in advance so it is verifiable that the categories were not fitted to the sample), the sampling seed, the sampled certificate column (from which the 75/100 re-derives), and the complete Lean proof development are released; raw per-company frequency tables are withheld per the source platform's terms, so the headline statistics are reproducible from the released derived data while the scrape itself is not redistributed. The proof artifact builds with `lake build` (zero `sorry`); the "standard axioms" claim is checkable, not merely asserted, by running `#print axioms` on the top-level theorems. This paper was written by a single human author with substantial AI assistance for drafting, statistical code, and the Lean formalization; all claims, numbers, and proofs were checked by the author, and the machine-checked results are, by construction, independently verifiable by re-running the proof development. We disclose this plainly: the paper is partly an artifact of the tools of its moment, and its central empirical claims stand or fall on the released data and the proof assistant, not on the means of production.

References

[1] S. Prashad. LeetCode Patterns. `https://seanprashad.com/leetcode-patterns/`, 2020.

[2] H. Cramér. *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[3] W. Bergsma. A bias-correction for Cramér's V and Tschuprow's T. *Journal of the Korean Statistical Society*, 42(3):323–328, 2013.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.

[5] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, 367–381, 2020.

[6] B. Nordhoff and P. Lammich. Dijkstra's shortest path algorithm. *Archive of Formal Proofs*, 2012.

[7] A. Armstrong, G. Struth, and T. Weber. Kleene algebra. *Archive of Formal Proofs*, 2013.

[8] D. J. Lehmann. Algebraic structures for transitive closure. *Theoretical Computer Science*, 4(1):59–76, 1977.

[9] S. Wimmer, S. Hu, and T. Nipkow. Monadification, memoization and dynamic programming. *Archive of Formal Proofs*, 2018.

[10] R. Bird and O. de Moor. *Algebra of Programming*. Prentice Hall, 1997.

[11] L. Heimes, D. Traytel, and J. Schneider. Formalization of an algorithm for greedily computing associative aggregations on sliding windows. *Archive of Formal Proofs*, 2020.

[12] T. Nipkow, J. Blanchette, M. Eberl, A. Gómez-Londoño, P. Lammich, C. Sternagel, S. Wimmer, and B. Zhan. *Functional Algorithms, Verified!* 2021. `https://functional-algorithms-verified.org`.

[13] T. Nipkow, M. Eberl, and M. P. L. Haslbeck. Verified textbook algorithms: A biased survey. In *Automated Technology for Verification and Analysis (ATVA)*, 2020.

A Family-to-scheme assignment rule

Each surface family is mapped to the scheme matching the *shape of its canonical accepted solution*: a single left-to-right pass carrying one accumulator → *streaming fold*; a recurrence that decomposes a structure into sub-results → *recursive decomposition (DP)*; iterative improvement toward a fixed point over a graph → *graph relaxation*; a monotone predicate located by halving → *bisection*; anything else (bespoke data structures, simulation, ad-hoc number theory, string matching) → *tail*. The rule is applied to the family, not tuned per problem; the per-problem Lean checks (Section 4) then test whether the assigned scheme actually solves each sampled problem.